Quand les données viennent d’ailleurs

Simulation, API, base locale ou flux temps réel

Si le ViewModel sait tout…

Lui confier aussi l'accès aux données semble pratique… au début.

God ViewModel

- État de l'UI
- Appels API
- SQL / NoSQL
- Cache / MQTT

Il grossit

Chaque source ajoute une nouvelle responsabilité.

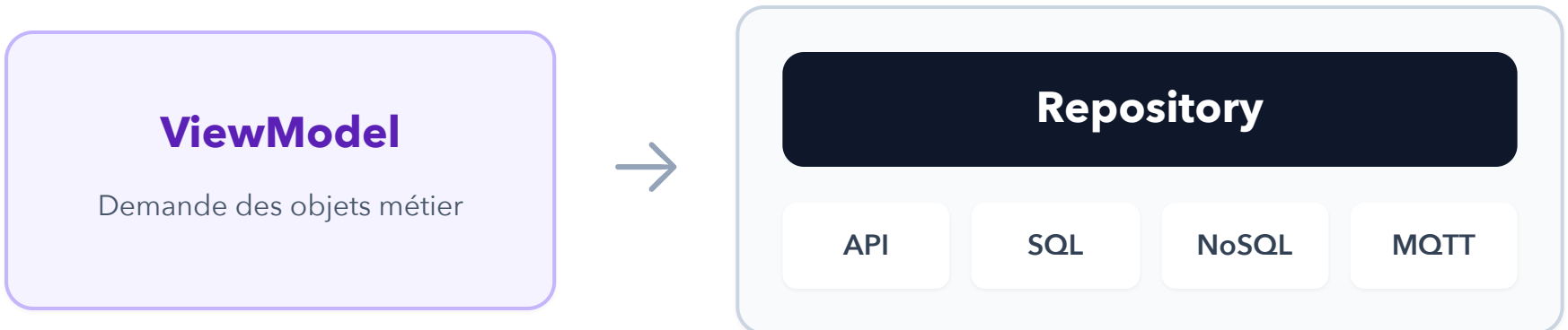
Il devient dépendant

Changer de technologie oblige à le modifier.

Le ViewModel doit savoir quelles données demander, pas où ni comment les récupérer.

Repository — une frontière stable

Le ViewModel exprime un besoin. Le Repository masque la manière d'y répondre.



Le ViewModel dépend d'un contrat stable, pas d'une implémentation technique.

Un contrat, une implémentation contrôlable

Le contrat

```
abstract interface class
    ItemRepository {
  Future<List<Item>> fetchAll();
}
```

Le ViewModel dépend uniquement de cette abstraction.

Le Fake Repository

- Données connues
- Latence simulée
- Erreur déclenchée à la demande

Quels paramètres permettraient de configurer ces comportements sans modifier la classe ?

Injecter immédiatement le Repository

```
class ScreenViewModel {
  ScreenViewModel({required ItemRepository repository})
      : _repository = repository;

  final ItemRepository _repository;
  List<Item> _items = [];

  List<Item> get items => List.unmodifiable(_items);

  Future<void> load() async {
    // À compléter : demander les données au Repository.
  }
}
```

Le Repository n'est pas une pièce isolée : il remplace immédiatement la liste écrite dans le ViewModel.

Un branchement volontairement provisoire

```dart
Future<void> main() async {
  final repository = /* choisir une implémentation */;
  final viewModel = /* injecter repository */;

  await viewModel.load();
  runApp(App(viewModel: viewModel)); // passage explicite
}
```

```
View → ViewModel → contrat → implémentation choisie
```

La chaîne fonctionne, mais deux problèmes deviennent visibles :

- rien ne s'affiche pendant l'attente ;
- une erreur empêche l'application de démarrer.

Étape 3 — Relier le Repository au ViewModel

Remplacer les données écrites dans le ViewModel par une source simulée asynchrone.

①

Définir le contrat

Décrire uniquement l'opération asynchrone nécessaire au ViewModel.

Le ViewModel dépend de l'interface

②

Créer le Fake

Pouvoir simuler des données, une latence et une erreur.

Aucune infrastructure réelle

③

Charger

Injecter le Repository puis remplacer les données statiques.

La View ne connaît pas le Repository

`View → ViewModel → Repository` Provisoirement : attendre le chargement avant `runApp`

Terminé lorsque le dashboard affiche les données du Fake et ne contient plus sa liste statique.

Concevoir l’état du ViewModel

Un chargement asynchrone peut-il être décrit par plusieurs variables indépendantes ?

Un ViewModel, trois informations d'état

Commençons par une représentation naïve de l'état :

```
class ScreenViewModel<T> {
  bool isLoading = false;
  String? errorMessage;
  List<T> values = [];
}
```

Ces propriétés peuvent évoluer indépendamment.

```
viewModel.isLoading = true;
viewModel.errorMessage = 'Erreur réseau';
viewModel.values = [/* résultat précédent */];
```

Si ces trois informations se contredisent, comment la View peut-elle choisir quoi afficher ?

Dans quel cas afficher chaque écran ?

```
if (viewModel.isLoading) {
  return const LoadingView();
}

final errorMessage = viewModel.errorMessage;
if (errorMessage ≠ null) {
  return ErrorView(message: errorMessage);
}

if (viewModel.values.isNotEmpty) {
  return ContentView(values: viewModel.values);
}

return const EmptyView();
```

Si chargement, erreur et contenu coexistent, la première condition gagne.

L'ordre des `if` détermine l'écran affiché, pas un état métier explicite.

Peut-on représenter la situation de l'écran par une seule valeur plutôt que résoudre les conflits dans la View ?

Une classe pour chaque état

```dart
abstract class AsyncState<T> {
  const AsyncState();
}

final class Loading<T> extends AsyncState<T> {
  const Loading();
}

final class Content<T> extends AsyncState<T> {
  const Content(this.values);
  final List<T> values;
}

final class Failure<T> extends AsyncState<T> {
  const Failure(this.message);
  final String message;
}
```

Une instance représente maintenant exactement un état : **Loading**, **Content** ou **Error**.

Un cas par défaut sans signification métier

```
// Switch expression
return switch (state) {
  Loading() ⇒ const LoadingView(),
  Content(:final values) ⇒ ContentView(values: values),
  Failure(:final message) ⇒ ErrorView(message: message),
  _ ⇒ const SizedBox.shrink(), // Cas imposé par une hiérarchie ouverte
};
```

Nous connaissons nos trois états, mais le compilateur considère que `AsyncState` pourrait avoir d'autres sous-types.

> **Une UI doit être déterministe :** à chaque état doit correspondre un rendu explicite. Le cas `_` contourne la vérification d'exhaustivité et peut masquer un état oublié.

Comment permettre au compilateur de vérifier que tous les états possibles sont traités ?

Fermer les états avec `sealed`

```dart
sealed class AsyncState<T> {
  const AsyncState();
}
```

```
AsyncState<T>
├── Loading<T>
├── Content<T>(values)
└── Failure<T>(message)
```

La hiérarchie devient fermée :

- un seul état est représenté à la fois ;
- le compilateur connaît tous les sous-types possibles ;
- un `switch` exhaustif n'a plus besoin de cas par défaut.

> **Alternative – Freezed :** il peut aussi générer cette union. Depuis Dart 3, `sealed`, le pattern matching et les `switch` exhaustifs sont natifs : nous privilégions donc Dart pour un simple état d'écran. Freezed reste utile si `copyWith`, l'égalité structurelle ou JSON sont également nécessaires.

Un rendu exhaustif dans la View

```
Widget renderState(AsyncState<Item> state) => switch (state) {
  Loading() => const LoadingView(),
  Content(values: []) => const EmptyView(),
  Content(:final values) => ContentView(values: values),
  Failure(:final message) =>
    ErrorView(message: message),
};
```

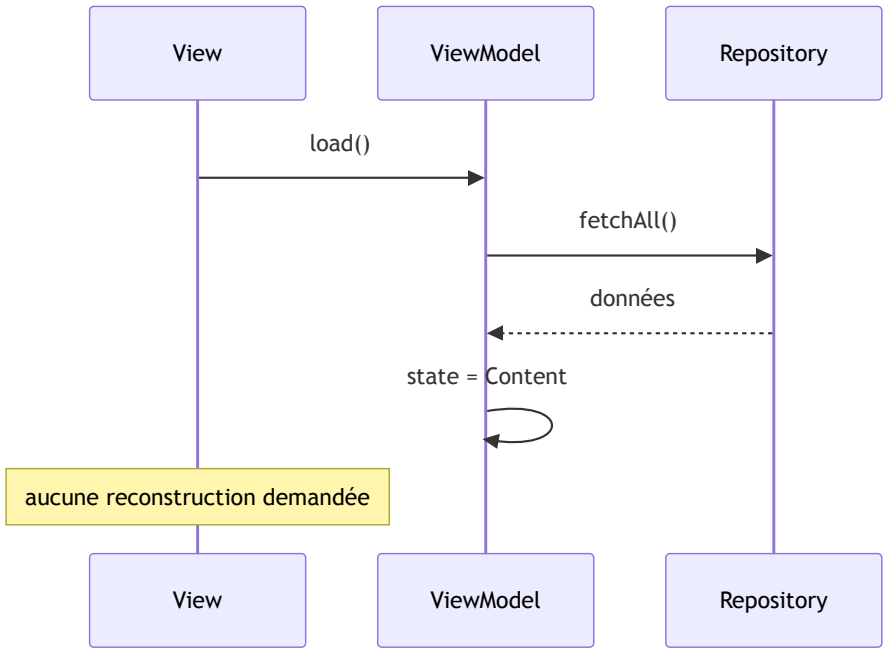
Si un nouvel état est ajouté et n'est pas traité, le compilateur signale le `switch` incomplet.

Le rendu est maintenant déterministe. Il reste à prévenir la View lorsque cet état change.

Rendre le ViewModel observable

Transmettre les changements d’état sans dépendre des widgets

Modifier l’état ne suffit pas



Modifier la propriété ne déclenche pas `build()` : il manque un signal vers la View.

`ChangeNotifier` en quatre opérations

`addListener(callback)`

L'observateur s'abonne.

`removeListener(callback)`

L'observateur se désabonne.

`notifyListeners()`

L'objet signale un changement.

`dispose()`

Le propriétaire libère l'objet.

État modifié → `notifyListeners()` → **Callbacks** → **Reconstruction**

Provider gérera l'abonnement et le cycle de vie. Le ViewModel déclenchera la notification.

Ajouter le signal au ViewModel

```
class ScreenViewModel<T> extends ChangeNotifier {
  ScreenViewModel({required Repository<T> repository})
      : _repository = repository;

  final Repository<T> _repository;

  AsyncState<T> _state = Loading<T>();
  AsyncState<T> get state => _state;

  void _setState(AsyncState<T> value) {
    _state = value;
    notifyListeners();
  }
}
```

L'état est remplacé en privé, puis les observateurs sont prévenus.

`ChangeNotifier` n'est pas un widget et ne nécessite aucun `BuildContext`. Provider se chargera plus tard d'exposer cette instance dans l'arbre.

La méthode `load()` doit maintenant enchaîner les états et signaler chacune de ses transitions.

Étape 4 — Rendre le ViewModel réactif

Mission

Faire publier au ViewModel un état cohérent à chaque étape du chargement.

Contraintes

- ne pas utiliser Provider ni construire de widget ;
- un seul objet décrit l'état courant de l'écran ;
- les cas possibles forment une hiérarchie fermée ;
- l'état est lisible mais non remplaçable depuis l'extérieur ;
- le ViewModel notifie après chaque transition.

À vous de concevoir les types d'état et d'implémenter les transitions.

> **Terminé lorsque :** un listener observe les états publiés sans Provider ni widget.